

Topic22: Project development

Developing Lead Management system from a scratch.

1. Overview of Project

2. Create a Lead entity

```
package com.example.app2;
import java.util.List;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
@Entity
public class Lead
{
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Integer id;
    public String name;
    private Long mobileNumber;
    private String emailId;
    private String status; //new, processing, rejected, closed
    @OneToMany(mappedBy = "lead", cascade = CascadeType.ALL)
    private List<Interaction> interactions;
    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Long getMobileNumber() {
        return mobileNumber;
    }
    public void setMobileNumber(Long mobileNumber)
    { this.mobileNumber = mobileNumber;
    }
    public String getEmailId() {
        return emailId;
    }
    public void setEmailId(String emailId)
    { this.emailId = emailId;
    }
    public String getStatus() {
        return status;
    }
    public void setStatus(String status) {
        this.status = status;
    }
}
```

```

        public List<Interaction> getInteractions()
        { return interactions;
        }
        public void setInteractions(List<Interaction> interactions)
        { this.interactions = interactions;
        }
    }

```

3. Create Interaction Entity

```

package com.example.app2;
import java.util.Date;
import java.util.List;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;

@Entity
public class Interaction
{
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Integer id;
    private String message;
    private Date date;
    @ManyToOne
    @JsonIgnore
    private Lead lead;
    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getMessage() {
        return message;
    }
    public void setMessage(String message)
    { this.message = message;
    }
    public Date getDate() {
        return date;
    }
    public void setDate(Date date) {
        this.date = date;
    }
    public Lead getLead() {
        return lead;
    }
    public void setLead(Lead lead) {
        this.lead = lead;
    }
}

```

4. Create LeadDTO

```

package com.example.app2;

public class LeadDTO {
    private Iterable<Lead> leads;
    public void setLeads(Iterable<Lead> leads) {
        this.leads = leads;
    }
    public Iterable<Lead> getLeads() {
        return leads;
    }
}

```

5. Create Repository

```

package com.example.app2;

import org.springframework.data.repository.CrudRepository;

public interface LeadRepository extends CrudRepository<Lead, Integer>{

}

```

6. Create Service class

```

package com.example.app2;
import java.util.Collections;
import java.util.List;
import java.util.Optional;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class LeadService
{
    @Autowired
    private LeadRepository leadRepository;

    public Lead save(Lead lead) {
        updateLeadToAllTheInteractions(lead);
        return leadRepository.save(lead);
    }
    public Lead update(Lead lead) {
        updateLeadToAllTheInteractions(lead);
        return leadRepository.save(lead);
    }
    public Lead read(Integer id) {
        Optional<Lead> optional = leadRepository.findById(id);
        Lead lead = optional.isPresent() ? optional.get() : null;
        return lead;
    }
    public LeadDTO readAll() {
        LeadDTO dto = new LeadDTO();
        Iterable<Lead> it = leadRepository.findAll();
        dto.setLeads(it);
        return dto;
    }

    private void updateLeadToAllTheInteractions(Lead lead)
    {
        if(lead.getInteractions() != null) {
            for(Interaction interacton : lead.getInteractions())
            {
                interacton.setLead(lead);
            }
        }
    }
}

```

7. Create Controller

```

package com.example.app2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController

```

```

@RequestMapping("lead")
@CrossOrigin
public class LeadController {
    @Autowired
    private LeadService leadService;

    @PostMapping
    public Lead save(@RequestBody Lead lead)
    { return leadService.save(lead);
    }

    @PutMapping
    public Lead update(@RequestBody Lead lead)
    { return leadService.save(lead);
    }

    @GetMapping("/{id}")
    public Lead read(@PathVariable Integer id)
    { return leadService.read(id);
    }

    @GetMapping
    public LeadDTO readAll() {
        return leadService.readAll();
    }
}

```

8. Configure the Leads which should load in the startup

```

package com.example.app2;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class App2Application implements
    CommandLineRunner{ @Autowired
    private LeadService leadService;

    public static void main(String[] args)
    { SpringApplication.run(App2Application.class, args);
    }

    @Override
    public void run(String... args) throws Exception
    { Lead lead1 = new Lead();
      lead1.setName("Mohan");
      lead1.setMobileNumber(9845098450L);
      lead1.setEmailId("m@mail.com");
      lead1.setStatus("new");
      leadService.save(lead1);

      Lead lead2 = new Lead();
      lead2.setName("Murali");
      lead2.setMobileNumber(9845098451L);
      lead2.setEmailId("mu@mail.com");
      lead2.setStatus("new");
      leadService.save(lead2);

      Lead lead3 = new Lead();
      lead3.setName("Swetha");
      lead3.setMobileNumber(9845098453L);
      lead3.setEmailId("s@mail.com");
      lead3.setStatus("new");
      leadService.save(lead3);
    }
}

```

9. React Routes configuration

10. App.js

11. LeadService.js

```

const url = 'http://localhost:9090/lead'
export const readAllLeads = () => {
  return axios.get(url);
}

export const readOneLead = (id) =>
{ return axios.get(url + "/" + id);
}

export const updateLead = (lead) =>
{ return axios.put(url, lead);
}

export const createLead = (lead) =>
{ return axios.post(url, lead);
}

```

12. Create.js

```

import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { createLead } from "../LeadService";
export default function Create() {
  const [name, setName] = useState('');
  const [mobileNumber, setMobileNumber] = useState('');
  const [emailId, setEmailId] = useState('');
  const [status, setStatus] = useState('new');
  const navigate = useNavigate();

  const handleSubmit = (e) =>
  { e.preventDefault();

    const lead = {name, mobileNumber, emailId, status};
    createLead(lead)
    .then((results) => {
      console.log('new lead added', results);
      navigate('/readAll')
    })
    .catch((err) => {
      console.log('error adding new lead', err)
    })
  }
  return (<
    <h1>Create</h1>
    Name: <input onChange={(e) => setName(e.target.value)} type='text' /> <br />
    Mobile Number: <input onChange={(e) => setMobileNumber(e.target.value)} type='text' /> <br />
    Email Id: <input onChange={(e) => setEmailId(e.target.value)} type='text' /> <br />
    <button onClick={handleSubmit} type='button'>Submit</button>    <button
    type='button'>Cancel</button>    <br />
    <Link to="/readAll">Read All</Link></>
  )
}

```

13. ReadAll.js

```

import { useEffect, useState } from "react";
import { readAllLeads } from "../LeadService";
import { Link } from "react-router-dom";
export default function ReadAll(){
  const [leads, setLeads] = useState([])
  const [searchBy, setSearchBy] = useState("name");
  const [filteredLeads, setFilteredLeads] = useState([])

```

```

const [nameAscOrder, setNameAscOrder] = useState(true);
const [mobileNumberAscOrder, setMobileNumberAscOrder] = useState(true);
const [emailIdAscOrder, setEmailIdAscOrder] = useState(true);
const [statusAscOrder, setStatusAscOrder] = useState(true);

useEffect(() =>
  { readAllLeads().then((results) => {
      setLeads(results.data.leads);
      setFilteredLeads(results.data.leads);
      console.log(results.data.leads)
    })
    .catch((error) =>
      { console.log(error)
    })
  }, [])

const sortByName = () => {
  var localLeads = [... filteredLeads];
  if(nameAscOrder){
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead1.name?.localeCompare(lead2.name)));
  }
  else{
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead2.name?.localeCompare(lead1.name)));
  }
  setNameAscOrder(!nameAscOrder);
}

const sortByEmailId = () => {
  var localLeads = [... filteredLeads];
  if(emailIdAscOrder){
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead1.emailId?.localeCompare(lead2.emailId)));
  }
  else{
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead2.emailId?.localeCompare(lead1.emailId)));
  }
  setEmailIdAscOrder(!emailIdAscOrder);
}

const sortByMobileNumber = () => {
  var localLeads = [... filteredLeads];
  if(mobileNumberAscOrder){
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead1.mobileNumber - lead2.mobileNumber));
  }
  else{
    setFilteredLeads(localLeads.sort((lead2, lead1) => lead1.mobileNumber - lead2.mobileNumber));
  }
  setMobileNumberAscOrder(!mobileNumberAscOrder);
}

const sortByStatus = () => {
  var localLeads = [... filteredLeads];
  if(statusAscOrder){
    setFilteredLeads(localLeads.sort((lead1, lead2) => lead1.status?.localeCompare(lead2.status)));
  }
  else{
    setFilteredLeads(localLeads.sort((lead2, lead1) => lead1.status?.localeCompare(lead2.status)));
  }
  setStatusAscOrder(!statusAscOrder);
}

const searchRecords = (text) =>
  { var localLeads = [...
    leads]; if(searchBy ===
    "name"){
      setFilteredLeads(localLeads.filter((lead) =>
lead.name?.toLowerCase().includes(text.toLowerCase())))
    }
    else if(searchBy === "mobileNumber"){
      setFilteredLeads(localLeads.filter((lead) => lead.mobileNumber?.toString().includes(text)))
    }
    else if(searchBy ===
      "emailId"){ setFilteredLeads(localLeads.filter((lead)
=>
lead.emailId?.toLowerCase().includes(text.toLowerCase())))
    }
  }

```

```

        else if(searchBy ===
            "status"){ setFilteredLeads(localLeads.filter((lead)
            =>
lead.status?.toLowerCase().includes(text.toLowerCase()))))
        }
    }
}

return (<
    Search By : <select onChange={(e) => setSearchBy(e.target.value)}>
        <option value="name">Name</option>
        <option value="mobileNumber">Mobile Number</option>
        <option value="emailId">Email Id</option>
        <option value="status">Status</option>
    </select>
    Search Text: <input type="text" onChange={(e) => searchRecords(e.target.value)}/>

    <h1>All Leads</h1>
    <table border='1'>
        <thead>
            <tr>
                <th>SNO</th>
                <th><a href="#" onClick={sortByName}>Name</a></th>
                <th><a href="#" onClick={sortByMobileNumber}>Mobile Number</a></th>
                <th><a href="#" onClick={sortByEmailId}>Email Id</a></th>
                <th><a href="#" onClick={sortByStatus}>Status</a></th>
                <th>Change Status</th>
            </tr>
        </thead>
        <tbody>
            {
                filteredLeads.map((lead, index) => <tr key={lead.id}>
                    <td>{index + 1}</td>
                    <td>{lead.name}</td>
                    <td>{lead.mobileNumber}</td>
                    <td>{lead.emailId}</td>
                    <td>{lead.status}</td>
                    <td>
                        <Link to={` /changeStatus/${lead.id}`}>Change Status</Link>
                    </td>
                </tr>)
            }
        </tbody>
    </table>
</>)
}

```

14. Sorting Records

```

const sortByName = () => {
    var localLeads = [... filteredLeads];
    if(nameAscOrder){
        setFilteredLeads(localLeads.sort((lead1, lead2) => lead1.name?.localeCompare(lead2.name)));
    }
    else{
        setFilteredLeads(localLeads.sort((lead1, lead2) => lead2.name?.localeCompare(lead1.name)));
    }
    setNameAscOrder(!nameAscOrder);
}

```

15. Search Records

```

const searchRecords = (text) =>
{
    var localLeads = [... leads];
    if(searchBy === "name"){
        setFilteredLeads(localLeads.filter((lead) =>
lead.name?.toLowerCase().includes(text.toLowerCase())))
    }
    else if(searchBy === "mobileNumber"){
        setFilteredLeads(localLeads.filter((lead) => lead.mobileNumber?.toString().includes(text)))
    }
}

```



```

        else if(searchBy ===
            "emailId"){ setFilteredLeads(localLeads.filter((lead)
=>
lead.emailId?.toLowerCase().includes(text.toLowerCase()))
        }
        else if(searchBy ===
            "status"){ setFilteredLeads(localLeads.filter((lead)
=>
lead.status?.toLowerCase().includes(text.toLowerCase()))
        }
    }
}

```

16. ChangeStatus.js

```

import { Link, useNavigate, useParams } from "react-router-dom";
import { readOneLead, updateLead } from "../LeadService";
import { useEffect, useState } from "react";

export default function ChangeStatus()
{
    const {id} = useParams();
    const navigate = useNavigate();
    const [lead, setLead] = useState({
        name:"",
        mobileNumber:"",
        emailId:"",
        status:"",
        interactions:[]
    });
    const [newInteraction, setNewInteraction] =
        useState({ date:"",
            message:""
        })
    useEffect(()=>{ readOneLead
        (id)
        .then((results)=>{ setLead(results.data)
        })
    }, [])

    const handleSubmit = (e) =>
    {
        e.preventDefault();
        setLead({... lead}, {id: id});
        console.log(lead)
        updateLead(lead)
        .then(results
            =>{ console.log(results.data)
        })
        .catch(error =>
            { console.log(error);
        });
        navigate('/readAll');
    }

    const addNewInteraction = () => {
        setLead({...lead, interactions: [...lead.interactions, newInteraction]});
        setNewInteraction({
            date:"",
            message:""
        })
    }

    return (<
        <h1>Change Status</h1>
        Name: {lead.name}<br/>
        Mobile Number: {lead.mobileNumber}<br/>
        Email Id: {lead.emailId}<br/>
        {lead.interactions?.length !== 0 && <h1>All Interactions</h1> && <table border='1'>
            <thead>

```

```

        <tr>
            <th>SNO</th>
            <th>Interaction Date</th>
            <th>Interaction Desc</th>
        </tr>
    </thead>
    <tbody>
        {lead.interactions.map((interaction, index) => <tr key={index}>
            <td>{index+1}</td>
            <td>{interaction.date}</td>
            <td>{interaction.message}</td>
        </tr>)}
    </tbody>
</table> }
<h4>Add New Interaction Message</h4>
Interaction Date: <input type="date"
                    value={newInteraction.date}
onChange={e=>setNewInteraction({...newInteraction, date: e.target.value})} /><br />
Interaction Desc: <input type="text"
                    value={newInteraction.message}
onChange={e=>setNewInteraction({...newInteraction, message: e.target.value})} /><br />
<button onClick={addNewInteraction} type='button'>Add Interaction</button><br />
Status:
<select value={lead.status}
        onChange={e=>setLead({...lead, status: e.target.value})}>
    <option value="new">New</option>
    <option value="processing">Processing</option>
    <option value="closed">Closed</option>
    <option value="rejected">Rejected</option>
</select> <br />
<button onClick={handleSubmit} type='button'>Update</button> <br />
<Link to='/readAll'>Back to All Leads</Link>

</>
}

```

17. Add New Interaction description

```

<h4>Add New Interaction Message</h4>
Interaction Date: <input type="date"
                    value={newInteraction.date}
onChange={e=>setNewInteraction({...newInteraction, date: e.target.value})} /><br />
Interaction Desc: <input type="text"
                    value={newInteraction.message}
onChange={e=>setNewInteraction({...newInteraction, message: e.target.value})} /><br />
<button onClick={addNewInteraction} type='button'>Add Interaction</button><br />

```